

Python Zero to Projects

Chapter 8

Dictionaries

Complete Study Notes

Topics: Creating Dictionaries -- Key Requirements -- Membership Operators

Dictionary Methods -- Looping -- Nested Dicts -- Dict Comprehension

Section 1: Creating Dictionaries and CRUD Operations

In Chapter 7 you learned about sets and discovered that `{}` on its own creates an empty **dictionary**, not an empty set. An empty set always requires `set()`. Now, in Chapter 8, that fact takes on its full meaning: `{}` is the entry point for every dictionary you will ever create.

A dictionary stores data as **key-value pairs**. Each key acts as a unique label, and each value is the data attached to that label. Think of a dictionary like a real-world contact card: the field names (name, phone, email) are the keys, and the actual data is the values.

1.1 The Empty Dictionary vs. The Empty Set

Example: empty dict vs empty set

```
# {} creates an empty DICTIONARY (not an empty set!)
empty_dict = {}
print(type(empty_dict))    # <class 'dict'>

# An empty SET requires set()
empty_set = set()
print(type(empty_set))     # <class 'set'>

# A set with values still uses {}
my_set = {1, 2, 3}         # set -- no colons, just values
my_dict = {"a": 1}         # dict -- colon separates key from value
```

NOTE: The presence of at least one colon (:) inside {} signals to Python that you are creating a dictionary, not a set. {} with no colon = empty dict. set() = empty set.

1.2 Creating a Dictionary with Key-Value Pairs

Example: dictionary literal

```
# Multi-line style (preferred for readability)
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}
print(student)
# {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Single-line style (fine for short dicts)
point = {"x": 3, "y": 7}
print(point)    # {'x': 3, 'y': 7}

# Mixed value types are allowed
profile = {"username": "alice99", "level": 5, "active": True}
print(profile)
```

Figure 1: A dictionary maps each key (left column) to one value (right column).

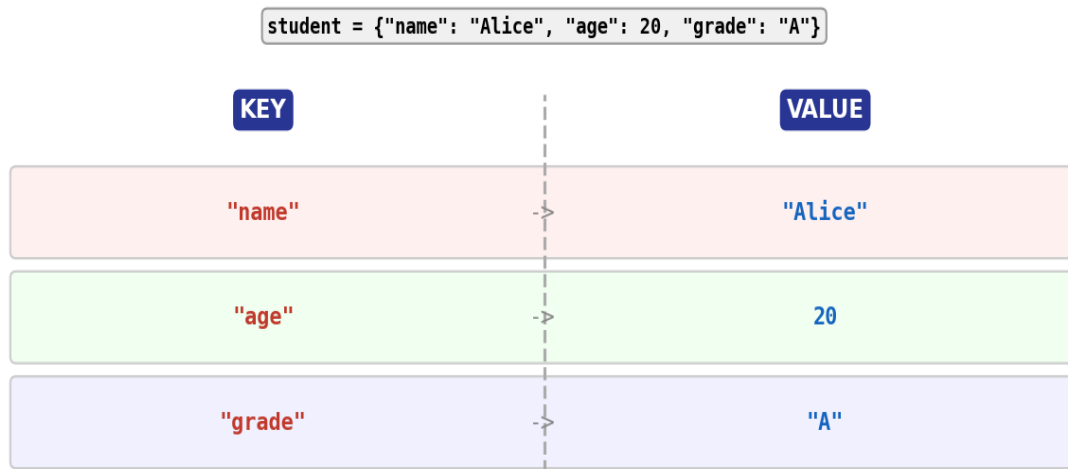


Figure 1 -- `student = {"name": "Alice", "age": 20, "grade": "A"}`

1.3 The dict() Constructor

Example: `dict()` constructor

```
# Using dict() with keyword arguments
student = dict(name="Alice", age=20, grade="A")
print(student)
# {'name': 'Alice', 'age': 20, 'grade': 'A'}

# dict() from a list of (key, value) tuples -- connects back to Chapter 6!
pairs = [("name", "Alice"), ("age", 20), ("grade", "A")]
student = dict(pairs)
print(student)
# {'name': 'Alice', 'age': 20, 'grade': 'A'}
```

CHAPTER LINK: Chapter 6 (Tuples): `dict()` from a list of tuples is a natural application of Chapter 6. Each (key, value) tuple in the list becomes one dictionary entry. You will see this pattern again with `.items()` and `zip()` in Chapter 10.

1.4 CRUD Operations on a Dictionary

CRUD stands for **Create, Read, Update, Delete** -- the four fundamental operations on any data store. Python dictionaries support all four cleanly with simple, readable syntax.

CREATE -- Add a new key-value pair

Create: assigning to a new key

```
student = {"name": "Alice", "age": 20}

# Assign to a NEW key to create it
student["grade"] = "A"
print(student)
```

```
# {'name': 'Alice', 'age': 20, 'grade': 'A'}
```

READ -- Access a value by its key

Read: `[]` access vs `.get()`

```
# Direct access with [] -- fast, but raises KeyError if key is missing
print(student["name"])    # Alice

# Safe access with .get() -- returns None (or a default) if key is missing
print(student.get("name"))    # Alice
print(student.get("email"))    # None    (no crash!)
print(student.get("email", "N/A"))    # N/A    (custom default)
```

UPDATE -- Modify the value of an existing key

Update: reassign an existing key

```
student["age"] = 21    # same syntax as CREATE; key already exists
print(student["age"])    # 21
```

DELETE -- Four ways to remove key-value pairs

Delete: `del` / `pop()` / `popitem()` / `clear()`

```
student = {"name": "Alice", "age": 21, "grade": "A", "city": "Boston"}
```

```
# del -- removes the pair; KeyError if key does not exist
del student["city"]
print(student)    # {'name': 'Alice', 'age': 21, 'grade': 'A'}
```

```
# pop(key) -- removes AND returns the value; KeyError if key missing
age_val = student.pop("age")
print(age_val)    # 21
print(student)    # {'name': 'Alice', 'grade': 'A'}
```

```
# pop(key, default) -- safe version; no KeyError even if key absent
fax = student.pop("fax", "no fax")
print(fax)    # no fax    (key was not present, no crash)
```

```
# popitem() -- removes and returns the LAST inserted (key, value) tuple
student = {"name": "Alice", "grade": "A", "score": 95}
last = student.popitem()
print(last)    # ('score', 95)
print(student)    # {'name': 'Alice', 'grade': 'A'}
```

```
# clear() -- removes ALL pairs; dict becomes empty {}
student.clear()
print(student)    # {}
```

WARNING: `del my_dict[key]` raises a `KeyError` if the key does not exist. Use `pop(key, default)` when you are unsure whether the key is present.

TIP: dict.pop(key) removes by KEY and returns the VALUE. list.pop(index) from Chapter 5 removes by INDEX. They share a name but work differently -- see the Common Mistakes section at the end of these notes.

NOTE: popitem() removes the LAST inserted pair (Python 3.7+ maintains insertion order). It is useful when you need to process and remove items one at a time without knowing the key names.

Section 2: Dictionary Key Requirements -- Hashability

Not every Python object can be a dictionary key. Two rules apply: (1) keys must be **unique** within a dictionary, and (2) keys must be **hashable** (i.e., immutable).

2.1 Rule 1 -- Keys Must Be Unique (Last Value Wins)

Example: duplicate keys

```
# Duplicate keys: Python keeps only the LAST value assigned
scores = {"math": 90, "math": 85, "math": 92}
print(scores)    # {'math': 92}    <-- only the last value survives

# No error is raised; the earlier values are silently overwritten
```

WARNING: Python does NOT warn you about duplicate keys -- it silently drops the earlier values. Watch for this when building dictionaries programmatically or copying from other data sources.

2.2 Rule 2 -- Keys Must Be Hashable (Immutable)

In Chapter 7 you learned that set elements must be hashable, because Python uses a hash table internally. The exact same rule applies to dictionary keys. A **hash** is a fixed-size integer computed from an object's value. Mutable objects (those that can change after creation) cannot be hashed reliably.

Types that CAN be dictionary keys (hashable / immutable)

Example: valid key types

```
# Strings (immutable -- Chapter 4)
d = {"name": "Alice", "city": "Boston"}

# Integers and floats
d = {42: "the answer", 3.14: "pi", 0: "zero"}

# Booleans (True and False are hashable integers)
d = {True: "yes", False: "no"}
print(d[True])    # yes

# Tuples containing only hashable elements (Chapter 6!)
grid = {(0, 0): "origin", (1, 2): "point A", (3, 4): "point B"}
print(grid[(1, 2)])    # point A
```

CHAPTER LINK: Chapter 6 (Tuples): Tuples are immutable, so they are hashable and can be used as dictionary keys. A tuple like (row, col) or (lat, lon) is a very common key in grid-based or geographic data structures.

Types that CANNOT be dictionary keys (unhashable / mutable)

Example: invalid key types -> TypeError

```

# Lists are MUTABLE -> UNHASHABLE -> cannot be keys (Chapter 5!)
try:
    bad = {[1, 2, 3]: "point"}
except TypeError as e:
    print(e)
# TypeError: unhashable type: 'list'

# Dicts are also mutable -> unhashable
try:
    bad = {"x": 1}: "value"}
except TypeError as e:
    print(e)
# TypeError: unhashable type: 'dict'

# Sets are also unhashable (Chapter 7!)
try:
    bad = {{1, 2, 3}: "set-key"}
except TypeError as e:
    print(e)
# TypeError: unhashable type: 'set'

# SOLUTION: convert a list to a tuple to make it a valid key
my_list = [1, 2, 3]
good = {tuple(my_list): "converted!"}
print(good)    # {(1, 2, 3): 'converted!'}

```

CHAPTER LINK: SYNTHESIS -- Chapters 5, 6, and 7: Lists (Ch 5) are mutable -> unhashable -> cannot be dict keys. Tuples (Ch 6) are immutable -> hashable -> CAN be dict keys. Sets (Ch 7) introduced hashability for set elements; that same concept now defines which types are valid dictionary keys. Strings (Ch 4) are immutable -> hashable -> always valid keys.

Section 3: len() and Membership Operators (in / not in)

3.1 len() on a Dictionary

len() counts the number of key-value **pairs** in a dictionary, not the total number of individual items. Each pair counts as one.

Example: len() on a dict

```
student = {"name": "Alice", "age": 20, "grade": "A"}
print(len(student))    # 3    (three key-value pairs)

empty = {}
print(len(empty))      # 0
```

3.2 'in' Checks KEYS by Default -- Not Values!

This is one of the most important and most commonly misunderstood behaviours of Python dictionaries. When you use the **in** operator directly on a dictionary, Python checks only the **keys**. It does **not** check values.

Example: 'in' checks KEYS; .values() checks VALUES

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# 'in' checks KEYS -- this is the default behaviour
print("name" in student)    # True    ("name" is a KEY)
print("Alice" in student)   # False   ("Alice" is a VALUE, not a key!)
print("email" in student)   # False   (this key does not exist)

# 'not in' is the opposite
print("email" not in student) # True

# -- To check for a VALUE, use .values() --
print("Alice" in student.values()) # True
print(20      in student.values()) # True
print("B"     in student.values()) # False
```

WARNING: CRITICAL TRAP: 'Alice' in student evaluates to False because 'in' checks keys, not values. To search for a value, always write 'Alice' in student.values(). Chapter 9 will use 'key in dict' extensively in if-statements -- understanding this distinction is essential.

NOTE: Checking 'key in dict' is O(1) -- near-instant -- because Python uses the key's hash to find it directly. This is the same speed advantage that set membership testing had in Chapter 7.

Section 4: Essential Dictionary Methods

4.1 keys(), values(), and items()

Example: keys(), values(), items()

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# .keys() -- returns all keys as a view object
print(student.keys())
# dict_keys(['name', 'age', 'grade'])

# .values() -- returns all values as a view object
print(student.values())
# dict_values(['Alice', 20, 'A'])

# .items() -- returns all (key, value) pairs as a view object
print(student.items())
# dict_items([('name', 'Alice'), ('age', 20), ('grade', 'A')])

# Convert to a list when you need indexing
key_list = list(student.keys())
print(key_list[0])    # name
print(key_list[1])    # age
```

NOTE: dict_keys, dict_values, and dict_items are view objects. They automatically reflect any changes made to the dictionary after they were created. They are not lists, but you can wrap them with list() when you need to index or slice them.

4.2 get() -- Safe Value Access with an Optional Default

get(key) returns the value for that key, or None if the key is missing -- without raising a KeyError.
get(key, default) lets you specify a custom fallback value.

Direct [] Access (Risky)	.get() Access (Safe)
<pre>#1: Create dict student = {"name": "Alice"} #2: Key EXISTS -> works fine print(student["name"]) # Output: Alice #3: Key MISSING -> program CRASH! print(student["email"]) # KeyError: 'email'</pre>	<pre>#1: Create dict student = {"name": "Alice"} #2: Key EXISTS -> works fine print(student.get("name")) # Output: Alice #3: Key MISSING -> safe return! print(student.get("email", "N/A")) # Output: N/A (no crash!)</pre>
KeyError crashes the program!	Returns None or default -- no crash!

Figure 2: Direct [] access raises KeyError on a missing key; .get() returns None or your chosen default value safely.

Example: get() with and without a default

```
student = {"name": "Alice", "age": 20}

# Key EXISTS -> returns the value normally
print(student.get("name"))          # Alice

# Key MISSING -> returns None (no KeyError!)
print(student.get("email"))         # None

# Key MISSING -> returns custom default
print(student.get("email", "not set")) # not set
print(student.get("phone", 0))       # 0

# The dict is NOT changed by .get()
print(student)  # {'name': 'Alice', 'age': 20} (unchanged)
```

4.3 update() -- Merge Another Dict or Key-Value Pairs

Example: update()

```
student = {"name": "Alice", "age": 20}

# Merge a second dictionary -- existing keys are UPDATED; new keys ADDED
extras = {"grade": "A", "age": 21}  # 'age' will be overwritten!
student.update(extras)
print(student)
# {'name': 'Alice', 'age': 21, 'grade': 'A'}

# update() also accepts direct keyword arguments
```

```
student.update(email="alice@school.com", year=2)
print(student)
# {'name': 'Alice', 'age': 21, 'grade': 'A',
#  'email': 'alice@school.com', 'year': 2}
```

4.4.setdefault() -- Get and Set in One Step

setdefault(key, default) is a two-in-one method: if the key **exists**, it returns its current value (no change to the dict). If the key is **missing**, it inserts key: default into the dict and returns the default. The dict IS changed in the missing case.

Example: setdefault()

```
student = {"name": "Alice"}

# Key EXISTS: returns existing value, dict is UNCHANGED
name = student.setdefault("name", "Unknown")
print(name)      # Alice
print(student)   # {'name': 'Alice'} (not overwritten)

# Key MISSING: inserts default, dict IS changed
email = student.setdefault("email", "not set")
print(email)     # not set
print(student)   # {'name': 'Alice', 'email': 'not set'}
```

TIP: setdefault() is very useful when building a dict of lists. Instead of checking 'if key not in d' first, write: d.setdefault(key, []).append(value) -- all in one line, no extra if-statement needed.

Section 5: Looping Through Dictionaries

There are three main patterns for iterating over a dictionary: over keys, over values, and over key-value pairs together. The `.items()` pattern with tuple unpacking (from Chapter 6) is the most powerful and most commonly used in real code.

5.1 Plain for Loop -- Iterates Over Keys

Example: plain for loop (yields keys)

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# A plain for loop over a dict yields KEYS
for key in student:
    print(key)

# Output:
# name
# age
# grade

# Use the key inside the loop to access the value
for key in student:
    print(key, "->", student[key])
# name -> Alice
# age -> 20
# grade -> A
```

5.2 Looping with `.keys()` and `.values()`

Example: `.keys()` and `.values()`

```
student = {"name": "Alice", "age": 20, "grade": "A"}

# .keys() -- explicit; same result as a plain for loop
for key in student.keys():
    print(key)
# name / age / grade

# .values() -- yields values directly, no key needed
for value in student.values():
    print(value)
# Alice / 20 / A
```

5.3 Looping with `.items()` and Tuple Unpacking (Most Pythonic)

`.items()` returns each entry as a (key, value) tuple. Combined with **tuple unpacking** from Chapter 6, you get both the key and value in one clean step -- the standard Pythonic way to loop a dictionary.

Example: `.items()` with tuple unpacking

```
student = {"name": "Alice", "age": 20, "grade": "A"}
```

```

# Tuple unpacking with .items() -- Chapter 6 in action!
# Each iteration yields a tuple ('name', 'Alice'), then ('age', 20) etc.
# Python unpacks it into the two loop variables: key and value
for key, value in student.items():
    print(f"{key}: {value}")
# Output:
# name: Alice
# age: 20
# grade: A

# Practical example: act on specific value types
for key, value in student.items():
    if isinstance(value, str):
        print(f" {key} is a string -> {value.upper()}")
# name is a string -> ALICE
# grade is a string -> A

```

CHAPTER LINK: Chapter 6 (Tuples): 'for key, value in student.items()' uses tuple unpacking. .items() returns (key, value) tuples, and Python automatically unpacks each into the two loop variables. This is the same mechanism as 'a, b = (1, 2)' from Chapter 6. Chapter 10 (Loops) will revisit this pattern with enumerate() and zip().

Section 6: Nested Dictionaries

Dictionaries can contain other dictionaries as values, can contain lists as values, and lists can contain dictionaries. These nested structures let you represent rich, real-world data like student records, product catalogs, or configuration files.

6.1 Dictionary Containing Dictionaries (dict of dicts)

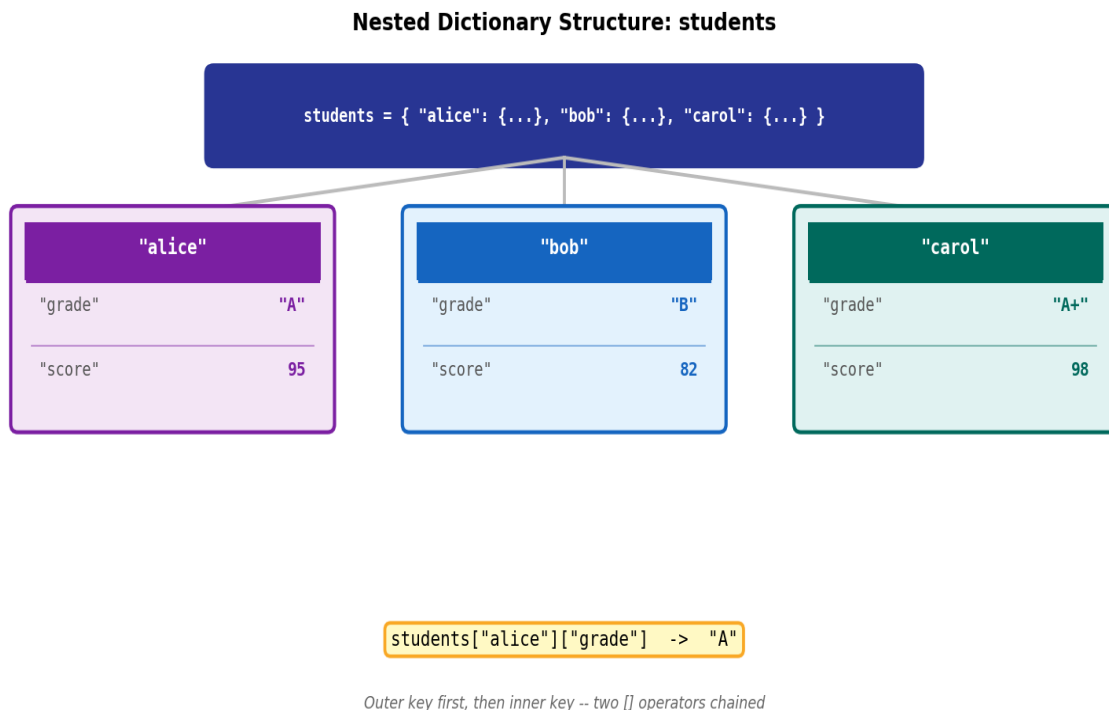


Figure 3: 'students' is an outer dict; each value is an inner dict with grade and score. Access chains two [] operators.

Example: dict of dicts

```
# Outer key = student name; value = inner dict with their details
students = {
    "alice": {"grade": "A", "score": 95},
    "bob": {"grade": "B", "score": 82},
    "carol": {"grade": "A+", "score": 98},
}

# Access nested value: outer_key -> inner_key (two [] operators)
print(students["alice"]["grade"])    # A
print(students["bob"]["score"])      # 82

# Modify a nested value
students["alice"]["score"] = 97
print(students["alice"])             # {'grade': 'A', 'score': 97}
```

```
# Add a new field to one inner dict
students["carol"]["rank"] = 1
print(students["carol"])    # {'grade': 'A+', 'score': 98, 'rank': 1}

# Loop through nested dict with .items()
for name, info in students.items():
    print(f"{name}: grade={info['grade']}, score={info['score']}")
# alice: grade=A, score=97
# bob:   grade=B, score=82
# carol: grade=A+, score=98
```

6.2 Dictionary with List Values

Example: dict with list values

```
inventory = {
    "fruits": ["apple", "banana", "orange"],
    "veggies": ["carrot", "broccoli"],
    "count": 5
}

# Access the list value
print(inventory["fruits"])    # ['apple', 'banana', 'orange']

# Index into the list
print(inventory["fruits"][1]) # banana

# Append to the list (lists are mutable even inside a dict!)
inventory["fruits"].append("grape")
print(inventory["fruits"])    # ['apple', 'banana', 'orange', 'grape']

# Loop over all fruits
for fruit in inventory["fruits"]:
    print(" -", fruit)
```

6.3 List of Dictionaries

Example: list of dicts

```
# Common pattern: a list of records, each record is a dict
students = [
    {"name": "Alice", "grade": "A"},
    {"name": "Bob",   "grade": "B"},
    {"name": "Carol", "grade": "A+"},
]

# Access by list index first, then dict key
print(students[0]["name"])    # Alice
print(students[2]["grade"])   # A+

# Loop through the list of dicts
```

```
for student in students:
    print(f"{student['name']} got grade {student['grade']}")
# Alice got grade A
# Bob got grade B
# Carol got grade A+

# List comprehension to filter (connects Ch 5 + Ch 8!)
top = [s["name"] for s in students if s["grade"] == "A"]
print(top)    # ['Alice']
```


Section 7: Dictionary Comprehension

Dictionary comprehension is a compact way to build a new dictionary from an iterable in a single expression. It follows the same family pattern as list comprehension (Chapter 5) and set comprehension (Chapter 7), but produces key-value pairs instead.

7.1 Syntax Overview

Syntax overview

```
# General pattern:
# { key_expression : value_expression for item in iterable }

# Comparison with the other comprehension types:
# List comprehension: [ expr for item in iterable ]
# Set comprehension: { expr for item in iterable }
# Dict comprehension: { key : value for item in iterable }
#                      ^   ^
#                      The colon between key and value is what
#                      makes Python treat this as a dict, not a set.
```

7.2 Examples

Example: dict comprehension patterns

```
# Example 1: Square each number from 1 to 5
squares = {x: x**2 for x in range(1, 6)}
print(squares)
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Example 2: Flip keys and values
original = {"name": "Alice", "city": "Boston"}
flipped = {value: key for key, value in original.items()}
print(flipped)
# {'Alice': 'name', 'Boston': 'city'}

# Example 3: Map words to their lengths
words = ["hello", "world", "python"]
word_lengths = {word: len(word) for word in words}
print(word_lengths)
# {'hello': 5, 'world': 5, 'python': 6}

# Example 4: Filter with a condition (preview of Chapter 10 loops!)
# Only include even numbers from 1..10 and their squares
evens_sq = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(evens_sq)
# {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

NOTE: Dict comprehension is introduced here as a preview of the shorthand syntax. Chapter 10 (Loops) will cover iteration in full depth, at which point comprehensions will feel completely natural. For

now, focus on reading and understanding the examples. List (Ch 5), set (Ch 7), and dict (Ch 8) comprehensions all share the same 'for item in iterable' core structure.

Section 8: Common Beginner Mistakes

Mistake 1: KeyError When Using [] on a Missing Key

Mistake 1: KeyError

```
student = {"name": "Alice"}

# WRONG -- raises KeyError; program crashes
# print(student["email"])      # KeyError: 'email'

# CORRECT -- use .get() for keys that might not exist
print(student.get("email"))    # None      (no crash)
print(student.get("email", "N/A")) # N/A      (custom default)

# Or guard with 'in' first (Chapter 9 preview)
if "email" in student:
    print(student["email"])
else:
    print("email field is missing")
```

Mistake 2: Assuming 'in' Checks Values

Mistake 2: 'in' checks keys, not values

```
student = {"name": "Alice", "age": 20}

# WRONG -- this checks KEYS, not values!
print("Alice" in student)      # False   (surprising!)

# CORRECT -- use .values() to search values
print("Alice" in student.values()) # True

# WRONG -- checking for a value that happens to share a name with a key
print("name" in student)       # True -- but this only tells you
                                # "name" is a KEY, not a value
```

Mistake 3: Using a List as a Dictionary Key

Mistake 3: list as key -> TypeError

```
# WRONG -- lists are unhashable; cannot be dict keys
try:
    d = {[1, 2, 3]: "point"}
except TypeError as e:
    print(e)
# TypeError: unhashable type: 'list'

# CORRECT -- convert the list to a tuple first (tuples are hashable!)
coords = [1, 2, 3]
d = {tuple(coords): "point"}
print(d[(1, 2, 3)])    # point
```

Mistake 4: Modifying a Dictionary While Iterating Over It

Mistake 4: modifying dict during iteration

```
scores = {"math": 90, "english": 75, "science": 88}

# WRONG -- changing dict SIZE during iteration raises RuntimeError
# for key in scores:
#     if scores[key] < 80:
#         del scores[key] # RuntimeError: dictionary changed size

# CORRECT -- iterate over a COPY of the keys list
for key in list(scores.keys()): # list() takes a snapshot of keys
    if scores[key] < 80:
        del scores[key]

print(scores) # {'math': 90, 'science': 88}
```

WARNING: Never add or delete keys while iterating over a dictionary directly. Always iterate over `list(my_dict.keys())` or `list(my_dict.items())` if you need to modify the dict inside the loop.

Mistake 5: Confusing dict.pop() with list.pop()

Mistake 5: dict.pop(key) vs list.pop(index)

```
# dict.pop(KEY) -- argument is a KEY; returns VALUE
student = {"name": "Alice", "age": 20}
removed = student.pop("age") # pass the KEY "age"
print(removed) # 20 (the VALUE is returned)
print(student) # {'name': 'Alice'}

# list.pop(INDEX) -- argument is an INDEX; returns the ELEMENT
scores = [90, 85, 78]
removed = scores.pop(1) # pass the INDEX 1
print(removed) # 85 (the ELEMENT at position 1)
print(scores) # [90, 78]

# Summary:
# dict.pop() -> argument = KEY (e.g., "name", "age")
# list.pop() -> argument = INDEX (e.g., 0, 1, -1)
```

Looking Ahead: Chapters 9 and 10

Chapter 9 -- Conditional Statements

Chapter 9 introduces `if / elif / else`. The `'in'` operator and `.get()` pair naturally with conditions to write safe, expressive logic involving dictionaries:

Preview: dicts with `if/elif/else` (Chapter 9)

```
# Pattern 1: check for a key before accessing it
if "email" in student:
    print(student["email"])
else:
    print("No email on file")

# Pattern 2: .get() as a conditional default
status = student.get("status", "inactive")
if status == "active":
    print("Student is currently enrolled")

# Pattern 3: look up a category with a fallback
role_label = {"admin": "Administrator", "mod": "Moderator"}
user_role = "guest"
display = role_label.get(user_role, "Standard User")
print(display)    # Standard User
```

Chapter 10 -- Loops

Chapter 10 covers `for` and `while` loops in depth. You will revisit looping through dictionaries with `.items()` and also meet `zip()` (for combining two iterables) and `enumerate()` (for adding a counter to any loop):

Preview: dicts with `zip` and `enumerate` (Chapter 10)

```
# Preview: zip() to build a dict from two parallel lists
keys   = ["name", "age", "grade"]
values = ["Alice", 20, "A"]
student = dict(zip(keys, values))
print(student)
# {'name': 'Alice', 'age': 20, 'grade': 'A'}

# Preview: enumerate() adds an index counter to .items()
for i, (key, value) in enumerate(student.items()):
    print(f" Field {i + 1}: {key} = {value}")
# Field 1: name = Alice
# Field 2: age = 20
# Field 3: grade = A
```

Chapter 8 Summary and Quick Reference

Creating dicts: Use {} with key:value pairs, or dict(). {} alone = empty dict (not empty set -- use set() for that). dict() accepts keyword args or a list of (key, value) tuples.

CRUD operations: Create: d[key] = value (new key). Read: d[key] (KeyError if missing) or d.get(key, default) (safe). Update: d[key] = new_value (existing key). Delete: del d[key] -- pop(key) -- popitem() (last pair) -- clear() (all).

Key requirements: Keys must be unique (last value wins on duplicates) and hashable. Valid: strings, ints, floats, booleans, tuples. Invalid: lists, dicts, sets (TypeError: unhashable type).

len() and membership: len(d) counts key-value pairs. 'key in d' checks KEYS only. 'val in d.values()' checks values. Key lookup is O(1) using hashing.

Methods: d.keys() / d.values() / d.items() -> view objects. d.get(k, default) -> safe read. d.update(other) -> merge (existing keys overwritten). d.setdefault(k, default) -> get-or-insert.

Looping: for k in d -- iterates keys. for v in d.values() -- iterates values. for k, v in d.items() -- both (uses tuple unpacking from Ch 6). Never add/remove keys during iteration; use list(d.keys()) copy.

Nested structures: d[outer][inner] for dict of dicts. d[key].append(val) for dict of lists. lst[i][key] for list of dicts.

Dict comprehension: {key_expr: val_expr for item in iterable} builds a dict in one line. Add 'if condition' at the end to filter items.

End of Chapter 8 Notes. Next up: Chapter 9 -- Conditional Statements.